

Context Management and Language Memoization: Dynamic Programming for Recursive Language-Model Agents

Adarsh Vatsa

Draft – April 2026

Abstract

Long-context language-model systems are often framed as search or retrieval problems. A more precise framing is that the agent is solving a context management problem under a fixed token budget. Recursive Language Models (RLMs) formalize one way to let a model inspect, decompose, and recursively call itself over snippets of an oversized context. However, this does not exhaust the design space: a well-built agent runtime with token-budget feedback, addressable context, isolated subagents, compaction, and self-reprompting can implement the same general strategy. This note argues that the more important missing layer is not recursion itself, but reusable recursive execution: a dynamic-programming-style memoization system for language-model subproblems. Instead of merely caching final answers, such a system should store structured intermediate work, source spans, dependency trails, confidence metadata, and reuse modes that allow future agents to replay, verify, compose, narrow, or rule out prior work. We also describe the first concrete implementation in this repository: a small structural memoization layer supporting exact replay, negative memoization, compositional coverage over context intervals, and JSON persistence.

1 Introduction

Long-context agents are increasingly asked to answer questions over inputs that exceed the context window of the underlying language model. One response is to build longer-context models. Another is to treat the full context as external to any single model call and require the agent to decide what subset of context should be inspected at each step.

The second view makes the underlying systems problem clear. The agent’s job is not simply to “search” the corpus. Its job is to manage scarce attention. It must decide when to delegate, when to compact, when to inspect a region, when to spawn a new instance of itself, and when the evidence it has collected is sufficient to answer.

This paper-style note develops that argument and then motivates a stronger cache abstraction for recursive agents. The proposed abstraction is not ordinary semantic caching of the form

$$\text{query} + \text{context} \rightarrow \text{answer}.$$

It is closer to dynamic programming over language-model subproblems: once a recursive subproblem has been solved over a context region, future calls should be able to reuse that work exactly, partially, compositionally, or as search guidance.

2 Recursive Context Management

Consider an agent given a question Q and a context C that is many times larger than its usable context window. The general solution is an iterative context-management loop:

1. Detect that C exceeds the usable prompt budget.
2. Choose a context-management move: delegate slices to subagents, compact state, inspect a selected region, or ask a new instance of itself with selected context and state.
3. Receive compact, structured outputs.
4. Decide whether the evidence is sufficient to answer Q .
5. If not, repeat with a better-targeted prompt or context selection.

This loop does not require one particular scaffold. It requires an orchestrator with feedback and control: approximate token counts, addressable context, isolated subagents, structured returns, and the ability to construct new prompts for future model instances.

3 RLM as a Formalization

Zhang, Kraska, and Khattab’s Recursive Language Models (RLMs) formalize one version of this loop. Their abstraction treats long prompts as part of an external environment and lets the model programmatically examine, decompose, and recursively call itself over snippets of the prompt [1].

This is a useful formal substrate, but it is not a fundamentally different class of operation from a well-built long-context agent runtime. A capable runtime with token-budget feedback, addressable context, isolated subagents, compaction, and self-reprompting can implement the same general strategy.

The distinction can be summarized as follows:

- RLM is a disciplined substrate for recursive context management.
- A strong agent runtime can be functionally equivalent if it exposes the same control over context selection, delegation, and compaction.
- The deeper unsolved problem is not merely whether an agent can recurse over context, but whether the system can avoid re-solving work it has already done.

4 Search as Candidate Generation

A model attending over a bounded chunk can often outperform classical search inside that chunk. It can resolve paraphrase, interpret local evidence, and synthesize answers in ways BM25 or vector similarity cannot.

This does not imply that search should be discarded. Classical search and indexing remain valuable as cheap candidate generators and context-addressing mechanisms. Their purpose is not to reason better than the model. Their purpose is to cheaply propose where model attention should be spent.

Useful low-cost signals include token counts, chunk identifiers, source spans, file or page meta-data, headings, lexical matches, vector candidates, cached summaries, and prior subagent results.

The target architecture is therefore index-assisted recursive attention: cheap indexing proposes candidate regions; bounded model calls read those regions; the orchestrator reasons over compact returns.

5 From Semantic Cache to Language Memoization

The current repository implements a useful semantic cache. It stores answers for query-context pairs, scopes reuse by source or dataset identity, verifies semantic equivalence before replay, and persists cache state across runs.

However, ordinary answer caching is too coarse for recursive agents. A future recursive call may not need the same final answer. It may need a supporting fact, a local summary, a proof fragment, a pointer to a likely region, a record that some region was already ruled out, or a partial aggregate over a subset of the context.

The stronger abstraction is a memoized language artifact:

Field	Meaning
task	Subproblem or question solved
context_scope	Document, chunk, span, or set of spans
result	Answer, fact, summary, decision, or pointer
evidence	Source spans supporting the result
dependencies	Subcalls used to produce the result
confidence	Confidence score or verifier status
reusable_as	Replay, fact, hint, aggregate, or ruled-out region

This shifts the system from binary cache hit/miss behavior to a reuse planner.

6 Reuse Modes

A memoized recursive system should support several reuse modes:

- Replay.** The same task has already been solved over the same context scope, so the prior result can be returned directly.
- Verify.** A prior result likely applies, but a cheap verifier or deterministic check should confirm that reuse is safe.
- Compose.** A larger task can reuse solved subregions and only call models for missing or ambiguous regions.
- Narrow.** Prior work identifies where the answer likely lives.
- Rule out.** Prior work identifies regions where the answer likely does not live.
- Aggregate.** Prior partial answers can be combined into a larger answer.

The model’s role changes accordingly. Instead of attending over all context from scratch, model calls become semantic operators for memoized work:

- Are two tasks equivalent?

- Does an old result entail what the new query needs?
- Does an old context span overlap enough to reuse?
- Can partial results compose into the larger answer?
- Is the prior result trustworthy enough to skip a model call?

7 A DP-Shaped Solver

The target solver has the structure of dynamic programming, but with semantic matching and verification in place of exact symbolic equality:

```
Solve(question Q, context C):
  if memo has an exact solved entry for Q over C:
    return it

  if memo has useful partial entries for Q over regions of C:
    reuse those entries
    identify missing or ambiguous regions
  else:
    identify candidate regions with indexes, summaries, or decomposition

  for each missing region:
    call a bounded subagent with Q and that region
    store the subagent result as a memo entry

  aggregate partial results
  verify answer and evidence
  store Solve(Q, C)
  return answer
```

This is different from ordinary semantic caching because the goal is not only answer replay. The goal is work avoidance and search guidance throughout a recursive reasoning process.

8 Relationship to the Current Repository

The existing implementation already contains several pieces needed for this direction:

- `source_chunk_hash` protects direct query reuse over exact source contexts.
- `data_scope_hash` protects retrieval-cache reuse across different document sets.
- The LLM Sniper verifies semantic equivalence before replaying a cached answer.
- Knowledge extraction begins moving from answer blobs toward fact-level reuse.
- Persistence allows work to survive across runs.

The architectural shift is:

cache answers $\longrightarrow$ memoize recursive subproblem work.

This requires first-class intermediate artifacts, evidence spans, dependency trails, verifier meta-data, and explicit reuse modes.

9 Concrete V0 Implementation

The repository now includes a first implementation of this idea in `language_memoization.py`. This module is deliberately smaller than the full semantic-cache system. It does not call an LLM, compute embeddings, or try to solve fuzzy semantic equivalence. Instead, it implements the deterministic substrate that a semantic layer can build on. The main `SemanticCacheController` now owns a `MemoStore`, persists it with the existing cache state, and exposes a controller-level `memoized_subproblem()` method for recursive workflows.

The v0 goal is:

Given a task and an addressable context interval, determine which parts of the work can be replayed, which parts have already been ruled out, which hints are available, and which intervals still require model calls.

9.1 Core Types

TaskSpec. `TaskSpec` describes the subproblem being solved. It contains the natural-language prompt, a task type, an output contract, and optional constraints. V0 task matching is exact after normalization: whitespace is collapsed and text is lowercased. This is intentionally conservative.

ContextScope. `ContextScope` is a half-open interval:

`[start, end)`

inside a corpus and document. The unit can be `chunk`, `char`, `token`, or any caller-defined coordinate system. A scope also carries an optional `content_hash`; if two scopes have different non-empty hashes, they are treated as incompatible. This prevents memoized work from one version of a document being reused against another.

MemoEntry. `MemoEntry` stores one reusable unit of model work. It contains the task, scope, result, evidence spans, dependencies, confidence, verifier status, and one or more reuse labels. The initial reuse labels are:

Label	Meaning
<code>exact_answer</code>	Can directly answer the same task/scope
<code>supporting_fact</code>	Can support a future answer
<code>search_hint</code>	Points future work toward a likely region
<code>aggregation_component</code>	Can compose into a larger aggregate
<code>ruled_out</code>	This region was already checked and failed

MemoStore. `MemoStore` is the in-memory table. It supports adding answer entries, adding negative entries, planning reuse, and saving/loading JSON state.

9.2 Planner Semantics

The central method is:

```
plan = store.plan_reuse(task, scope)
```

The planner follows four deterministic rules:

1. If an **exact_answer** exists for the same normalized task and exact same compatible scope, return an exact replay plan.
2. Otherwise, collect all entries with the same normalized task and a compatible overlapping scope.
3. Split them into reusable answer/aggregate entries, negative **ruled_out** entries, and **search_hint** entries.
4. Treat answer, aggregate, supporting-fact, and ruled-out entries as coverage. Subtract their intervals from the requested scope to produce **missing_scopes**.

This gives the orchestrator a concrete action plan. If **missing_scopes** is empty, the requested scope is fully covered by prior work. If not, the orchestrator only needs to spawn subagents for the missing intervals.

9.3 Controller Integration

The controller integration has two pieces.

First, **SemanticCacheController.memoized_subproblem()** lets callers solve scoped subproblems through the memo planner. The caller provides a **solver(task, scope)** function. In production, that solver can call a subagent or model. In tests, it is a deterministic function. The controller calls the solver only for **missing_scopes**, stores each new result as a **MemoEntry**, and then stores a composed exact entry for the larger scope when coverage is complete.

Second, **search()** is memo-first. It mirrors legacy final-answer cache rows and legacy knowledge triples into memo entries, then checks the memo graph for exact scoped replay before falling back to document retrieval and answer synthesis. The older data structures remain loadable, but they are compatibility inputs rather than separate reuse systems.

The structural flow is:

```
plan = memo_store.plan_reuse(task, requested_scope)

if plan.has_exact_replay:
    return prior_answer

for scope in plan.missing_scopes:
    result = solver(task, scope)
    memo_store.add_answer(...) or memo_store.add_negative(...)

if all intervals are covered:
    memo_store.add_answer(task, requested_scope, composed_answer)
```

9.4 DuckDB Backend

The default `MemoStore` is an in-memory dictionary with JSON persistence. That is appropriate for tests and small local runs, but it scans entries linearly. The repository therefore includes `DuckDBMemoStore`, an optional local backend for larger benchmark runs.

DuckDB is used narrowly: it stores durable structured memo rows, raw context chunks, dependency edges, and performs fast candidate lookup. Python still performs interval subtraction and reuse planning. This keeps the database role simple and keeps planner behavior identical across memory and DuckDB backends. The model is not given unrestricted SQL execution. Instead, the controller exposes structured operations such as lexical chunk search and range fetch, then passes bounded candidate packets to the model.

The important table columns are:

Column	Purpose
<code>entry_id</code>	Stable memo entry identity
<code>task_signature</code>	Normalized task lookup key
<code>corpus_id, document_id</code>	Corpus and document isolation
<code>scope_start, scope_end</code>	Interval lookup
<code>unit</code>	Coordinate system, e.g. chunk or char
<code>content_hash</code>	Document-version guard
<code>reusable_as_json</code>	Reuse labels
<code>entry_json</code>	Full round-trippable memo entry
<code>context_chunks</code>	Raw chunk text keyed by corpus, document, and index

The key lookup is:

```
SELECT entry_json
FROM memo_entries
WHERE task_signature = ?
      AND corpus_id = ?
      AND document_id = ?
      AND unit = ?
      AND content_hash = ?
      AND scope_start < requested_end
      AND scope_end > requested_start
```

This returns only memo entries for the same task and overlapping context interval. The planner then classifies them as exact replay, reusable coverage, negative coverage, or hints.

Usage is intentionally parallel to the in-memory store:

```
from language_memoization import DuckDBMemoStore

store = DuckDBMemoStore("memo.duckdb")
store.add_answer(task, scope, "$12.4M")
plan = store.plan_reuse(task, larger_scope)
store.close()
```

When `solve_with_memo()` receives chunked context and the memo store is DuckDB-backed, the controller also persists those chunks in a `context_chunks` table. The practical retrieval surface is:

```

store.fetch_context_range(
    corpus_id="my-corpus",
    document_id="doc-1",
    start=0,
    end=4,
)

store.search_context_chunks(
    "launch codeword",
    corpus_id="my-corpus",
    document_id="doc-1",
)

controller.context_candidate_packets("launch codeword", scope=scope)

```

This gives the orchestrator an addressable context substrate without turning retrieval into raw model-authored SQL. A future planner can ask for structured operations such as “fetch this range” or “search chunks for these terms”, while the implementation remains parameterized and auditable.

This backend is useful when memo entries become numerous enough that JSON round-trips and linear scans are no longer comfortable, but a full server database such as Postgres would be operationally excessive.

9.5 Example

Suppose an agent must find a financial metric over chunks 0 through 10. The memo store already knows that chunks 0 through 2 were checked and did not contain the metric, and that chunks 5 through 7 contain a useful partial answer. The planner returns only the uncovered intervals:

```

from language_memoization import (
    ContextScope, MemoStore, REUSE_AGGREGATION_COMPONENT, TaskSpec
)

task = TaskSpec(
    prompt="Find the Q1 ARR",
    task_type="financial_metric_lookup",
    output_contract="short factual answer",
)

def scope(start, end):
    return ContextScope(
        corpus_id="finance",
        document_id="model-a",
        start=start,
        end=end,
        unit="chunk",
        content_hash="document-version-1",
    )

```



```

store = MemoStore()
store.add_negative(task, scope(0, 2), reason="metric not found")
store.add_answer(
    task,
    scope(5, 7),
    "$12.4M appears in the KPI table",
    reusable_as=(REUSE_AGGREGATION_COMPONENT,),
)

plan = store.plan_reuse(task, scope(0, 10))

# plan.missing_scopes is:
# [ContextScope(start=2, end=5), ContextScope(start=7, end=10)]

```

The important point is that the system does not ask “do we have the final answer?” It asks “which parts of this recursive computation are already solved?”

9.6 Negative Memoization

Negative entries are first-class. If a subagent has already inspected a region and found nothing, that result is useful. It prevents future agents from spending attention on the same failed region. In search-heavy recursive tasks, this can be as valuable as caching positive facts.

For example, a `ruled_out` entry over chunks 0–2 counts as solved coverage for the same task. A later query over chunks 0–10 only needs to inspect the rest.

9.7 Current Clean Architecture

The current implementation is now memo-first. The older semantic-cache and knowledge mechanisms still exist for backward compatibility, but they no longer own independent reuse decisions in the main `search()` path. They are mirrored into memo entries:

- legacy final answers become `exact_answer` memo entries;
- legacy knowledge triples become `supporting_fact` memo entries;
- FAISS, lexical memo search, and DuckDB chunk search are candidate generators;
- exact replay, partial composition, semantic verifier reuse, evidence, and dependency edges are represented through the memo graph.

The practical boundary is `reuse_candidate_packets()`. It exposes the candidate generators as bounded packets while keeping the memo graph as the single source of truth for reuse.

```

packet = controller.reuse_candidate_packets(query, scope=scope)

packet["memo_entries"]    # reusable work
packet["context_chunks"]  # raw evidence candidates

```

Q3 is used when language judgment is required: solving scoped tasks, aggregating partial memo entries, or verifying semantic reuse. It is not the cache, and it does not write raw SQL.

10 Research Positioning

The clean distinction from RLM is not that recursive context management is new. It is that recursive context management benefits from persistent, scoped, semantic memoization.

Layer	Role
RLM	Recursively navigate and process huge context
Memoization	Remember and reuse recursive subproblem work
Provenance	Determine which subanswers are trustworthy
Planning	Use prior subanswers to guide future decomposition

Put differently:

RLM gives recursive execution. Language memoization gives reusable recursive execution.

11 Practical Next Steps

The current repository now has a practical MVP of the automatic path:

```
answer = controller.solve_with_memo(
    query,
    chunks,
    solver=local_q3_solver,
    document_id="synthetic-doc",
    chunk_size=2,
    aggregate_fn=local_q3_aggregator,
)
```

This method decomposes a document into chunk windows, checks the memo store for each scoped subproblem, calls the supplied solver only for missing windows, passes reusable partial answers to an aggregator, stores the full-scope answer, and replays that full answer on later calls. It also accepts a `reuse_verifier` callback. That verifier may approve reuse from a different prompt over the same scope, producing a new exact memo entry whose dependency points back to the source memo entry. This is the first safe step toward semantic dynamic programming: fuzzy reuse is possible, but only when a verifier explicitly allows it.

The live smoke test exercises this path with DuckDB and a local MLX Qwen model:

```
python scripts/live_dp_memo_smoke.py \
--reset-db \
--backend duckdb \
--duckdb-path benchmark_artifacts/live_dp_memo.duckdb
```

The benchmark scripts deliberately avoid small repository-level generation caps on the normal path. The `-max-tokens` and `-aggregate-max-tokens` flags remain available for explicit stress tests, but omitted flags leave generation length to the MLX backend or to `RLMS_MLX_MAX_TOKENS` if the operator sets it.

The expected shape is:

- first solve: two scoped model calls and one aggregation call;
- exact replay: zero scoped model calls and zero aggregation calls;
- verifier-approved semantic replay: zero solver and aggregation calls for a different prompt over the same scope;
- persistence: reopening the DuckDB file preserves exact replay.

The same DP memoization path also has a small NoLiMa fixture benchmark:

```
python scripts/run_dp_memo_nolima.py \
  --reset-db \
  --max-cases 2 \
  --max-haystacks 1 \
  --max-samples 4 \
  --lengths 250 \
  --depth-intervals 2 \
  --chunk-words 60 \
  --chunk-size 2
```

This benchmark uses the repository’s local NoLiMa needle and haystack fixtures. It writes predictions, bridge rows, a manifest, and a DuckDB memo store under `benchmark_artifacts/dp_memo_nolima/`. In the current smoke run, the four fixture samples cover two needles across two depths, and every sample is immediately replayed from DuckDB after the first solve. `-max-cases` and `-max-haystacks` define the source slice; `-max-samples` is only an early stop inside that generated slice.

A more direct benchmark for memoized reuse is the shared-context fixture:

```
python scripts/run_dp_memo_shared_context.py \
  --reset-db \
  --sentences-per-chunk 2 \
  --chunk-size 1
```

This benchmark asks several related questions over one document. The system first solves a reusable fact-extraction task over the document chunks and stores those facts in DuckDB. Each question is then answered by giving the local model compact memo candidate packets and asking it to select the useful entries. This tests the complementary path directly: cheap memo retrieval proposes candidates; the model reads structured memo content and chooses the answer. The current run answers all five fixture questions correctly and verifies zero-call replay for the memoized fact layer.

The DuckDB backend now also materializes the memo graph explicitly. Dependencies are mirrored into a `memo_edges` table with parent ID, child ID, edge type, timestamp, and metadata. The initial read APIs are:

- `graph_edges()`
- `children(entry_id)`
- `parents(entry_id)`

- `lineage(entry_id)`

This keeps the existing deterministic replay path unchanged while making the dependency graph auditable and queryable.

11.1 Current Instrumentation

The memo substrate now exposes telemetry at two levels.

First, every `MemoReusePlan` can report JSON-safe coverage telemetry:

- requested scope length;
- covered length;
- missing length;
- coverage ratio;
- covered intervals;
- missing scopes;
- exact, reusable, negative, and hint entry IDs;
- fragment-kind counts for all candidates and covering candidates;
- evidence-span and dependency-edge counts.

This matters because the benchmark should not only report whether the final answer was correct. It should report how much work the memo graph avoided. A typical row can now distinguish:

```
initial coverage: 0.00
final coverage:   1.00
reused windows:   2 / 3
coverage kinds:   supporting_fact=1, ruled_out_region=2
```

Second, the store exposes `stats()`:

- total memo entries;
- counts by fragment kind;
- counts by result type;
- counts by verifier status;
- counts by reuse mode;
- dependency edge count;
- evidence span count;
- average confidence.

The fragment kind is planner-facing. Instead of asking a model to infer the role of a memo row from loose text, the packet tells it whether the row is an `exact_answer`, `supporting_fact`, `aggregation_component`, `search_hint`, or `ruled_out_region`. This is a small schema choice, but it is central to the “split cache” framing: the value of the system depends on storing the right fragments, not just storing more answer strings. Because reuse-plan telemetry includes fragment-kind counts, an ablation can now distinguish coverage from exact answer replay, supporting facts, aggregation components, and negative memoization instead of reporting only one opaque coverage number.

Candidate packet text can still be bounded for prompt construction, but truncation is no longer silent. Memo packets include `result_chars`, `result_truncated`, `evidence_count`, and per-evidence `text_truncated` fields. Raw context packets include `text_chars` and `text_truncated`. Passing `None` for the relevant max-character argument disables truncation. This avoids a common failure mode in agentic systems: hidden context budgets silently removing the evidence the planner needed.

11.2 Long-Context Benchmark Path

The project now includes a NoLiMa workload inspector:

```
python scripts/inspect_long_context_workload.py \
  --needle-set-path benchmark_data/nolima/needlesets/needle_set.json \
  --haystack-dir benchmark_data/nolima/haystack/rand_shuffle_long \
  --lengths 32K,64K,128K,256K \
  --depth-intervals 4 \
  --max-cases 4 \
  --max-haystacks 1 \
  --chunk-words 1000 \
  --chunk-size 2
```

This script does not call a model. It estimates how many samples, chunks, solver windows, and cold model calls a benchmark slice implies. For the example slice above, the inspector reports 64 samples and 3,904 estimated cold model calls. The 64K, 128K, and 256K settings force recursive decomposition rather than direct prompting.

This is the correct benchmark direction for this repo because NoLiMa [2] minimizes literal overlap between the question and the needle. A vector or lexical retriever can still serve as a candidate generator, but it should not be able to solve the task by string matching alone. The memo layer should be evaluated on whether repeated domain-scoped calls progressively fill a reusable fragment graph that reduces future work.

The first official-data smoke surfaced a fragment-design issue. Asking every scoped chunk to produce the final answer caused over-filtering: the relevant chunk contained a local fact such as “Katie lives next to the Kiasma museum,” while the question asked which character had been to Helsinki. A direct-answer subcall returned `NOT_FOUND`. The better fragment was not a final answer, but a reusable evidence fragment. The runner now supports:

```
--solver-mode evidence
```

In this mode, scoped calls extract broad named-entity and relationship facts, and the aggregator performs the final inference. On the same official 8K NoLiMa smoke sample, this changed the answer

from an incorrect NoLiMa to the correct answer, *Katie*, while preserving the same memo behavior: 21 scoped calls, one aggregate call, full final coverage, and zero-call exact replay.

A broader 8K slice later exposed a sharper two-hop failure. A question asked which character had been to *Uusimaa*; the relevant slice said *Katie lives next to the Kiasma museum*. The first evidence extractor still over-filtered and returned `NOT_FOUND`, so the aggregator never received the bridge fact. The runner now makes evidence mode stricter about recall: if a slice contains any named person, place, landmark, title, code, or relationship, the extractor should emit a fact instead of `NOT_FOUND`. On the targeted failed sample this changed the answer back to *Katie*. The tradeoff is clear: recall-oriented fragments improve two-hop correctness but produce more supporting facts and higher latency.

The warm-start run over the same DuckDB store then answered the same official sample with zero scoped model calls and zero aggregation calls. This gives the minimal real benchmark lifecycle: cold execution builds the fragment graph; warm execution replays from it.

The next test used overlapping length sweeps. By opting into stable corpus, document, and source-hash identities, a 16K solve can become usable prefix work for a later 32K solve over the same source. In the 32K run, the planner started with 81 of 161 chunks already covered, skipped 40 windows, and called the model only for the expanded tail. The answer remained correct. This is the first practical partial-composition result: the larger task reused a solved subregion rather than replaying an identical full answer.

The same pattern held as the official slice grew:

Run	Initial coverage	Model calls	Aggregate calls	Correct
16K cold	0.000	41	1	yes
32K partial	0.503	41	1	yes
64K partial	0.502	81	1	yes
128K partial	0.501	161	1	yes
128K warm	1.000	0	0	yes
256K partial	0.579	234	1	yes
256K warm	1.000	0	0	yes

The 128K partial run reused 321 of 641 chunks from the existing memo graph, processed only the 320 missing chunk positions, and then stored a full-scope answer. A fresh-process warm replay over the same DuckDB file answered with zero solver calls, zero aggregation calls, and about 150 ms of wall-clock time. The 256K run extended the same pattern: it started with 641 of 1108 chunks covered, skipped 320 windows, solved 234 missing windows, answered correctly, and then warm-replayed in about 135 ms with zero model calls. An attempted 512K probe over the same single-haystack slice produced the same 1108 chunk scope and exact-replayed immediately. That is a useful benchmark limitation: once the selected haystack source is exhausted, increasing the requested length label no longer creates a harder workload. Larger experiments therefore need either larger source files or multiple haystacks/cases, not just a larger nominal length flag. This does not prove general agent quality, but it does show the intended substrate behavior: as a corpus grows in overlapping stages, prior scoped work remains usable.

11.3 What the Benchmark Does and Does Not Prove

The NoLiMa path is useful because it stress-tests a real failure mode: long contexts where literal overlap is weak and naive chunk answering can miss indirect evidence. It is not, by itself, a complete measure of real-world agent performance. The benchmark has a particular shape: a synthetic long

context, a hidden needle, a short answer, no evolving workspace, and no naturally repeated user workflow unless the experiment creates one.

This distinction matters for memoized RLMS. The central claim is not that the system should overfit a single long-context QA format. The claim is that repeated, domain-bounded knowledge work creates reusable intermediate subproblems:

- the same corpus is queried repeatedly;
- questions overlap without being identical;
- useful evidence fragments survive across tasks;
- old negative searches can prevent repeated dead ends;
- partial work over a smaller scope can become useful inside a larger scope.

A cold, single-shot benchmark mostly measures first-pass search. That can understate the value of a persistent memo graph, because the graph becomes more valuable as a workload continues. Conversely, real deployments introduce risks that synthetic benchmarks may understate: stale fragments, changed documents, permission boundaries, ambiguous user goals, and unsafe semantic reuse.

The right evaluation target is therefore a workload trace, not only a single question-answer pair. A good benchmark for this substrate should report:

- final accuracy;
- model calls, wall-clock time, and token cost;
- cold-start performance;
- warm replay performance;
- partial-overlap reuse across growing or changing corpora;
- stale-memory failure rate after document updates;
- which fragment kinds were actually reused.

In this framing, NoLiMa is one instrument in the evaluation suite. It helps surface fragment-design problems. The stronger evidence for this project comes from workload-style runs where the memo graph is filled once, then reused across related tasks or larger overlapping scopes.

11.4 Staleness and Scope Invalidation

Persistent memoization introduces a safety problem that ordinary one-shot benchmarks rarely test: old work can become stale. Content hashes prevent many accidental reuses, but a practical agent also needs an explicit operational control for document updates. The memo store now exposes:

```
store.invalidate_scope(scope, reason="document updated")
```

This rejects all non-rejected memo entries that overlap the supplied scope. The entries are not deleted. They remain available for audit and lineage, but exact replay, partial composition, and planner candidate generation ignore them. Invalidation also propagates through dependency parents by default: if a derived answer depends on a changed fragment, that derived answer is rejected even when its own scope does not directly overlap the edited chunk. This matters because a long-lived memo graph should behave like a versioned working memory, not an append-only bag of facts that are always trusted.

Passing a scope with an empty content hash invalidates compatible entries for that document range regardless of version. Passing a specific content hash only invalidates entries compatible with that version. This gives callers a direct way to handle document edits, corpus refreshes, or user corrections without destroying the historical record.

There are two useful versioning modes:

- Use strict content hashes when any document change should force a clean recomputation.
- Use stable or empty content hashes plus explicit `invalidate_scope()` when the system should reuse unaffected windows after a localized update.

The second mode is closer to a real mutable workspace. A regression test now covers this path: after chunk 1 changes, the old full answer and old chunk-1 fragment are rejected, chunks 0, 2, and 3 remain reusable, and the next solve calls the solver only for chunk 1.

The repository also includes a deterministic mutable-workspace benchmark:

```
python scripts/run_dp_memo_mutable_workload.py --reset-db
```

The trace solves a four-chunk operational runbook, updates one chunk, invalidates that scope, solves the updated version, and then reopens DuckDB for warm replay. The current run has the intended shape:

```
v1_model_calls: 4
invalidated_entries: 2
v2_model_calls: 1
v2_initial_coverage_ratio: 0.75
warm_model_calls: 0
```

This is a tiny benchmark, but it tests a qualitatively important workload pattern that single-shot long-context QA does not: persistent memory must remain useful across localized document updates.

11.5 Composition Fix

One implementation bug surfaced while using the substrate on the long-context path. The document-level `solve_with_memo()` loop used to skip a window only when that exact window had an exact replay entry. That missed a central dynamic-programming case: a larger memo fragment may already fully cover a smaller window.

The loop now skips any window whose reuse plan is complete. This allows a window to be avoided if it is covered by:

- an exact window answer;

- a larger solved region;
- supporting facts;
- an aggregation component;
- a ruled-out region.

This change preserves the old exact replay path but makes partial composition real in the automatic document solver.

Concrete next steps for the repository are:

1. Run longer official NoLiMa slices with Q3 and inspect coverage, reused-window ratio, answer accuracy, and wall-clock cost.
2. Extend `solve_with_memo()` from fixed chunk windows to structure-aware windows based on sections, records, pages, or functions.
3. Add verifier calls only when reuse would skip meaningful model work.
4. Add semantic task matching on top of the structural task signature.
5. Build ablations comparing no memoization, exact answer replay, semantic answer replay, fact reuse, and full memoized subproblem reuse.

12 Conclusion

Recursive context management is a powerful way to make fixed-context language models operate over inputs much larger than their context windows. But recursion alone does not prevent repeated work. The more durable systems contribution is a language memoization graph: a persistent, scoped, verified record of recursive subproblems and their reusable intermediate results.

Such a system would let an orchestrator solve a long-context task once and then reuse the resulting subproblem graph whenever later tasks overlap with that work.

References

- [1] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive Language Models. *arXiv preprint arXiv:2512.24601*, 2026. <https://arxiv.org/abs/2512.24601>.
- [2] Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schuetze. NoLiMa: Long-Context Evaluation Beyond Literal Matching. *International Conference on Machine Learning*, 2025. <https://arxiv.org/abs/2502.05167>.
- [3] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, and Juanzi Li. LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks. *ACL*, 2025. <https://longbench2.github.io/>.
- [4] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. InfiniteBench: Extending Long Context Evaluation Beyond 100K Tokens. *ACL*, 2024. <https://github.com/OpenBMB/InfiniteBench>.